

W5_hw1 Report

[1] Design and Implement 1-Bit and 4-Bit Full Adder

1. Objectives:

- (1) 1-bit full adder와 4-bit full adder가 어떤 원리로 작동하는지를 이해하고 파악한다.
- (2) 1-bit full adder를 dataflow, gate-level modeling 방법으로 소스코드를 구현한다.
- (3) 구현한 1-bit full adder를 이용해 structural modeling 방법으로 4-bit full adder를 구현한다.

2. Theoretical Approach:

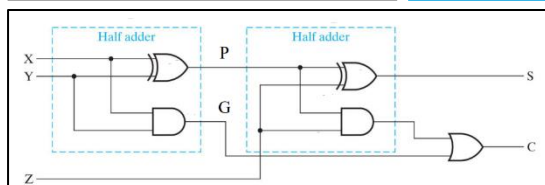
X	0	0	1	1
+Y	+0	+1	+0	+1
CS	00	01	01	10

1-bit half adder의 경우에는 두 inputs를 이용해 sum과 carry를 구하는 단순 계산의 방식이다. 왼쪽의 계산과 같이 단순히 이진수의 덧셈이 이루어지고 결과로 carry와

Z	0	0	0	0
X	0	0	1	1
+Y	+0	+1	+0	+1
CS	00	01	01	10
Z	1	1	1	1
X	0	0	1	1
+Y	+0	+1	+0	+1
CS	01	10	10	11

Inputs			Outputs	
X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

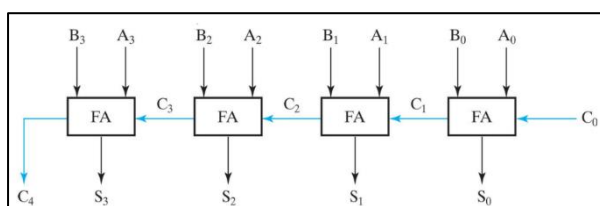
sum을 구하게 된다. 1-bit full adder의 adder의 경우에는 두 inputs와 이전 자리올림을 이용해 sum과 carry를 구하는 방식이다. 왼쪽의 계산처럼 inputs X, Y와 함께 carry Z까지 계산에 포함된다. 계산의 결과로 나타낸 sum, carry의 truth table이 왼쪽에 나타나 있다.



Carry와 sum을 Boolean expression으로 나타내면

$S = X \oplus Y \oplus Z$ $C = XY + (X \oplus Y)Z$ 이고 이를 왼쪽과 같이 gates로 나타낼 수 있다.

4-bit full adder는 1-bit full adder 4개를 사용하여 나타낼 수 있다. 처음 LSB부터 계산을 할 때



carry 0을 입력하여 첫 번째 1-bit full adder를 실행하고 발생하는 carry를 이용하여 나머지도 똑같이 수행해 sum과 carry out을 출력한다. 이를 diagram으로 도식화한 것이 왼쪽 그림에 해당된다.

3. Verilog Implementation:

(1) Dataflow Modeling of 1-Bit Full Adder :

//sum

```
assign sum = (a ^ b ^ cin);
```

//cout

```
assign cout = (a && b) || ((a ^ b) && cin);
```

1-Bit Full Adder의 Boolean expression은 $S = X \oplus Y \oplus Z$ $C = XY + (X \oplus Y)Z$ 이다. 이를 그대로 Logic Operations를 사용하여 Dataflow Modeling 소스코드를 구현하였다.

(2) Gate-Level Modeling of 1-Bit Full Adder :

//sum

```
xnor_gatelevel_gate xnor_1 (.a(a), .b(b), .out(xnor_out_1));
```

```
not_gate not_1 (.a(xnor_out_1), .out(not_out_1));
```

```
xnor_gatelevel_gate xnor_2 (.a(not_out_1), .b(cin), .out(xnor_out_2));
```

```
not_gate not_2 (.a(xnor_out_2), .out(sum));
```

//cout

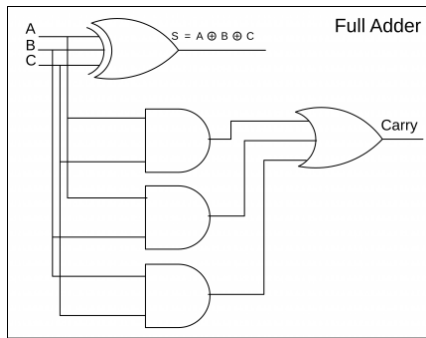
```
and_gate and_1 (.a(a), .b(b), .out(and_1_out));
```

```
and_gate and_2 (.a(a), .b(cin), .out(and_2_out));
```

```
and_gate and_3 (.a(b), .b(cin), .out(and_3_out));
```

```
or_gate or_1 (.a(and_1_out), .b(and_2_out), .out(or_1_out));
```

```
or_gate or_2 (.a(and_3_out), .b(or_1_out), .out(cout));
```



wire로 xnor_out이 2개, not_out이 1개가 있었기 때문에 sum을 도출할 때 모두를 이용하기 위해서는 앞의 input 2개를 먼저 xnor에 입력하고 나온 값을 inverting한 다음에 다시 carry와 xnor에 넣고 나온 out을 inverting하여 최종적으로 xor에 입력한 값과 같도록 하였다. Carry를 출력하는데 있어서는 and_out 3개, or_out 1개가 wire로 선언되었으므로 왼쪽의 그림 같이 gates를 배치하여 소스코드를 구현하였다.

(3) Structural Modeling of 4-Bit Full Adder :

```
input [3:0] a, b;
```

```
input cin;
```

```
output [3:0] sum;
```

```
output cout;
```

```
full_adder_gatelevel_module adder_1 (.a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout(cout_1));
```

```
full_adder_gatelevel_module adder_2 (.a(a[1]), .b(b[1]), .cin(cout_1), .sum(sum[1]), .cout(cout_2));
```

```
full_adder_gatelevel_module adder_3 (.a(a[2]), .b(b[2]), .cin(cout_2), .sum(sum[2]), .cout(cout_3));
```

```
full_adder_gatelevel_module adder_4 (.a(a[3]), .b(b[3]), .cin(cout_3), .sum(sum[3]), .cout(cout));
```

input에 4-bit a, b와 0값으로 입력되는 cin을 선언하고 그에 따른 output도 선언하였다. 4-Bit Full Adder는 1-Bit Full Adder 4개를 사용하여 구현해야 하기 때문에 미리 구현한 full adder of gate-level modeling을 이용하였다. Gate-level modeling으로 만든 1-Bit Full Adder를 submodule로 하여 총 4개의 adder를 사용하였다는 것을 알 수 있다. 최종적으로 4-Bit의 sum과 cout이 도출된다.

(4) Testbench of Modules :

```
for (count = 0; count <8; count = count +1)
```

```
    #10 {A,B,IN_FA_CIN} <= count;
```

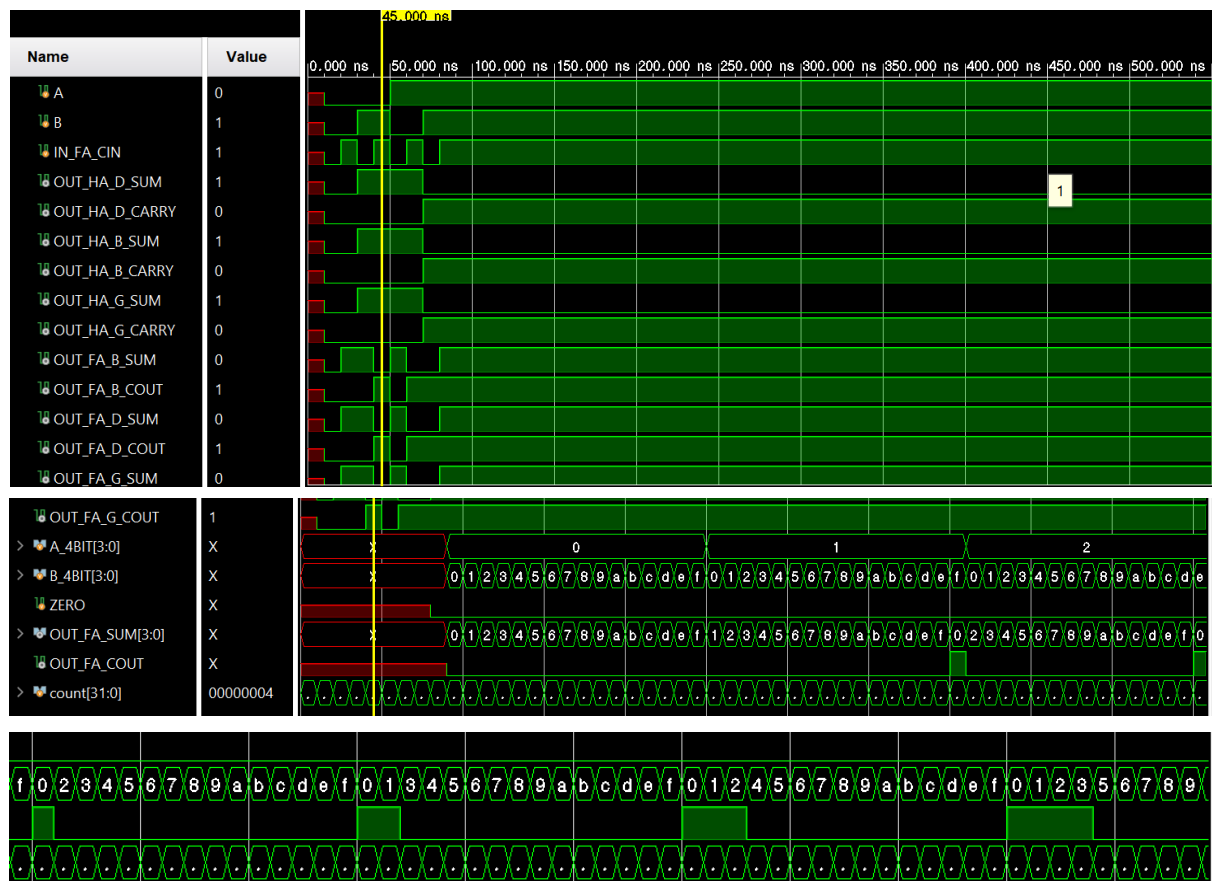
```
ZERO = 0;
```

```
for (count = 0; count <256; count = count +1)
```

```
    #10 {A_4BIT, B_4BIT} <= count;
```

Half-Adder와 1-Bit Full-Adder의 inputs인 A, B, FA_CIN에 10ns만큼의 delay를 주어 count가 1씩 증가하도록 설정했다. 3-bit이므로 총 80ns가 지나면 ZERO가 0으로 입력되고 다음 10ns 이후부터는 4-Bit Full-Adder에 입력되는 4-bit의 A, B inputs가 10ns마다 count가 1씩 증가하도록 설정했다.

4. Simulation :



Inputs			Outputs	
X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

우선 A, B, IN_FA_CIN이 000 ~ 111까지 10ns 간격으로 진행되고 이에 따른 결과값이 이미 예시로 주어졌던 behavioral modeling의 outputs과 같게 나타나고 있음을 확인할 수 있다. 산출되는 OUT_FA_SUM과 OUT_FA_COUT이 왼쪽의 Truth Table과 부합하고 있다. FA에 대한 Simulation의 결과가 세 modeling 모두 일치하므로 디자인 한 대로 되었다고 할 수 있다. 1-Bit FA가 모두 끝나고 ZERO와 4-Bit FA가 시작되고 있는데 이는 Testbench에서 설정한 값과 일치한다. Simulation이 진행되는 모습을 보면 A_4BIT가 0일 때는 COUT이 1인 때가 0번, 1일 때는 1번, 2일 때는 2번으로 계속 진행되고 있는데 이는 4-Bit FA의 Truth Table을 반영한 결과이다. 따라서 Simulation이 예상한 결과와 정확히 일치하고 있음이 증명되었다는 것을 확인할 수 있다.

는 모습을 보면 A_4BIT가 0일 때는 COUT이 1인 때가 0번, 1일 때는 1번, 2일 때는 2번으로 계속 진행되고 있는데 이는 4-Bit FA의 Truth Table을 반영한 결과이다. 따라서 Simulation이 예상한 결과와 정확히 일치하고 있음이 증명되었다는 것을 확인할 수 있다.

5. Conclusion :

1-Bit Full Adder를 Dataflow, Gate-Level Modeling의 방법을 통해 소스코드를 구현하고 구현한 1-Bit Full Adder를 submodule로 사용하여 Structural Modeling 방법을 통해 4-Bit Full Adder의 소스코드도 구현하였다. Count의 for loop를 이용해 test pattern을 입력하여 simulation을 시행한 결과 미리 주어졌던 1-Bit Full Adder의 Behavioral Modeling의 simulation 결과값과 정확히 일치하고 있음을 확인하였다. 또한 결과값이 Truth Table의 값을 정확히 반영하고 있는 것 또한 확인할 수 있었다. 4-Bit Full Adder의 simulation도 예상한 데이터를 그대로 반영하고 있음을 확인한 바로 modules에 대한 source code가 정확히 디자인 되었다고 말할 수 있다.